Friday — Voice & Emotional Journaling Assistant

This repository contains a focused codebase to upgrade your LiveKit `Friday` assistant into a voice-first emotional journaling assistant. It includes memory persistence, per-user journals, voice recording & transcript pipeline hooks, simple emotion extraction (via LLM prompt), mood logging, playback, and basic CLI entrypoint. Replace placeholders (API keys, model endpoints) with your actual credentials.

Project structure

```
friday-journal/
├── README.md
├── requirements.txt
├── .env.example
├── main.py                # Entrypoint worker
├── assistant.py           # Assistant class and session logic
├── memory.py              # Memory manager (per-user)
├── tools.py               # Tool functions (journal, mood, playback)
├── prompts.py             # Prompts for LLM calls
├── utils.py               # Helpers: audio save, emotion extraction helper
└── agent_memory.json      # sample memory file (generated at runtime)
```

README.md

````
# Friday — Voice & Emotional Journaling

This project upgrades your LiveKit voice assistant into a voice-first emotional
journaling assistant.

## Setup

1. Create a Python 3.10+ virtual environment.

```bash
python -m venv .venv
source .venv/bin/activate  # or .\.venv\Scripts\activate on Windows
pip install -r requirements.txt
````

1. Copy `.env.example` to `.env` and fill required keys (LIVEKIT, GOOGLE, OPENAI etc.).
````

2. Run locally:

```
python main.py
```

Notes

- This code uses LiveKit Agents and a Google Realtime LLM in the example—swap with your preferred model.
- `tools.py` contains journaling-specific tools and hooks. You can extend them to call external sentiment/tone APIs.

```

```

---

requirements.txt

```
python-dotenv
livekit
# Depending on your LiveKit package, adjust the name above
requests
pydub
python-dotenv
# Optional: if you use OpenAI for emotion detection
openai
```

.env.example

```
LIVEKIT_API_KEY=
LIVEKIT_API_SECRET=
GOOGLE_API_KEY=
OPENAI_API_KEY=
MEMORY_FILE=agent_memory.json
```

---

prompts.py

```
# prompts.py

```

```
AGENT_INSTRUCTION = """
You are Friday, a gentle voice-first emotional journaling assistant. Your role
is to help the user capture voice journals, tag emotions, summarize patterns,
and generate affirmations. Be empathetic, concise, non-judgmental, and
supportive.
"""

SESSION_INSTRUCTION = """
When the session starts, introduce yourself briefly and offer to help with a
voice journal, mood log, or reflection. Ask a gentle prompt if the user seems
quiet.
"""
```

---

memory.py

```
# memory.py
import json
import os
import datetime
from typing import Dict, Any

MEMORY_FILE = os.environ.get("MEMORY_FILE", "agent_memory.json")

DEFAULT_MEMORY = {"users": {}}


def load_memory() -> Dict[str, Any]:
    if os.path.exists(MEMORY_FILE):
        try:
            with open(MEMORY_FILE, "r") as f:
                return json.load(f)
        except Exception:
            return DEFAULT_MEMORY.copy()
    return DEFAULT_MEMORY.copy()


def save_memory(mem: Dict[str, Any]):
    with open(MEMORY_FILE, "w") as f:
        json.dump(mem, f, indent=2)


memory = load_memory()
```

```python
def ensure_user(user_id: str):
    if user_id not in memory["users"]:
        memory["users"][user_id] = {"journal": [], "mood_log": []}


def add_journal_entry(user_id: str, text: str, tags: list, intensity: int,
audio_path: str | None = None):
    ensure_user(user_id)
    entry = {
        "id": f"j_{len(memory['users'][user_id]['journal'])+1}",
        "date": datetime.date.today().isoformat(),
        "text": text,
        "tags": tags,
        "intensity": intensity,
        "audio": audio_path,
    }
    memory["users"][user_id]["journal"].append(entry)
    save_memory(memory)
    return entry


def add_mood_log(user_id: str, mood: str, intensity: int, note: str | None =
None):
    ensure_user(user_id)
    item = {"date": datetime.date.today().isoformat(), "mood": mood,
"intensity": intensity, "note": note}
    memory["users"][user_id]["mood_log"].append(item)
    save_memory(memory)
    return item


def get_recent_journals(user_id: str, n: int = 5):
    ensure_user(user_id)
    return memory["users"][user_id]["journal"][-n:]


def get_mood_summary(user_id: str):
    ensure_user(user_id)
    # simple summary: counts per tag and avg intensity
    journals = memory["users"][user_id]["journal"]
    tag_counts = {}
    total = 0
    for j in journals:
        total += j.get("intensity", 0)
        for t in j.get("tags", []):
            tag_counts[t] = tag_counts.get(t, 0) + 1
    avg_intensity = (total / len(journals)) if journals else 0
    return {"tag_counts": tag_counts, "avg_intensity": avg_intensity}
```

utils.py

```
# utils.py
import os
import uuid
from pydub import AudioSegment

AUDIO_DIR = "audio_clips"

os.makedirs(AUDIO_DIR, exist_ok=True)


def save_audio_bytes_bytes(raw_bytes: bytes, extension: str = ".wav") -> str:
    filename = f"{uuid.uuid4().hex}{extension}"
    path = os.path.join(AUDIO_DIR, filename)
    with open(path, "wb") as f:
        f.write(raw_bytes)
    return path


async def simple_emotion_extractor(openai_client, text: str) -> dict:
    """
    Lightweight emotion extraction using an LLM (OpenAI example). Returns tags
list and an intensity (1-10).
    Replace with your LLM call of choice.
    """
    # This is an example for OpenAI's API - adapt if you use a different model.
    prompt = (
        "Classify the primary emotions in the following journal entry.\n"
        "Return a JSON object with keys: tags (list of short labels), intensity
(1-10 integer).\n\n"
        f"Text:\n{text}\n\n"
        "Respond only with a JSON object."
    )
    try:
        resp = openai_client.ChatCompletion.create(
            model="gpt-4o-mini",  # replace as needed
            messages=[{"role": "user", "content": prompt}],
            temperature=0.0,
            max_tokens=200,
        )
        # resp parsing depends on client; adapt as necessary
        content = resp["choices"][0]["message"]["content"]
        import json
```

```
        parsed = json.loads(content)
        return parsed
    except Exception:
        # fallback naive extractor
        lower = text.lower()
        tags = []
        if any(w in lower for w in ["sad", "depressed", "unhappy"]):
            tags.append("sadness")
        if any(w in lower for w in ["happy", "grateful", "joy"]):
            tags.append("joy")
        if any(w in lower for w in ["angry", "frustrat"]):
            tags.append("anger")
        intensity = min(10, max(1, sum(1 for t in tags) * 3)) if tags else 5
        return {"tags": tags, "intensity": intensity}
```

---

tools.py

```
# tools.py
"""
Tools designed to be passed into the Assistant. They are lightweight wrappers
around memory and utils.
"""
from typing import Optional
import asyncio
from memory import add_journal_entry, add_mood_log, get_recent_journals,
get_mood_summary
from utils import save_audio_bytes_bytes, simple_emotion_extractor
import os
import json


async def handle_voice_journal(user_id: str, transcript: str, audio_bytes:
Optional[bytes], openai_client=None):
    audio_path = None
    if audio_bytes:
        audio_path = save_audio_bytes_bytes(audio_bytes, extension=".wav")

    # Emotion extraction
    emotion = {"tags": [], "intensity": 5}
    if openai_client:
        emotion = await simple_emotion_extractor(openai_client, transcript)
    else:
        emotion = await simple_emotion_extractor(None, transcript)
```

```
    tags = emotion.get("tags", [])
    intensity = int(emotion.get("intensity", 5))

    entry = add_journal_entry(user_id, transcript, tags, intensity, audio_path)
    return entry


async def handle_mood_log(user_id: str, mood: str, intensity: int = 5, note:
Optional[str] = None):
    return add_mood_log(user_id, mood, intensity, note)


async def list_journals(user_id: str, n: int = 5):
    return get_recent_journals(user_id, n)


async def summarize_insights(user_id: str):
    return get_mood_summary(user_id)
```

---

assistant.py

```
# assistant.py
from livekit.agents import Agent
from prompts import AGENT_INSTRUCTION
from livekit.plugins import google


class Assistant(Agent):
    def __init__(self, tools=None):
        super().__init__(
            instructions=AGENT_INSTRUCTION,
            llm=google.beta.realtime.RealtimeModel(voice="Aoede",
temperature=0.7),
            tools=tools or [],
        )
```

---

main.py

```
# main.py
import os
from dotenv import load_dotenv
```

```
from livekit import agents
from livekit.agents import AgentSession, RoomInputOptions, JobContext
from livekit.plugins import noise_cancellation

from assistant import Assistant
from prompts import SESSION_INSTRUCTION
from memory import memory
from tools import handle_voice_journal, handle_mood_log, summarize_insights,
list_journals
from utils import save_audio_bytes_bytes

load_dotenv()


async def entrypoint(ctx: JobContext):
    print("[Agent] Starting Friday Assistant session...")
    session = AgentSession()

    await session.start(
        room=ctx.room,
        agent=Assistant(tools=[]),
        room_input_options=RoomInputOptions(video_enabled=False,
noise_cancellation=noise_cancellation.BVC()),
    )

    await ctx.connect()
    print("[Agent] Connected.")

    # Compose greeting
    greeting = SESSION_INSTRUCTION
    await session.generate_reply(instructions=greeting)

    # Hook: This is an example handling loop; actual LiveKit events should use
session.on_* handlers
    # Here we only show a conceptual pattern of how you might process a voice
journal.


if __name__ == "__main__":
    agents.cli.run_app(agents.WorkerOptions(entrypoint_fnc=entrypoint))
```

Notes & Next Steps

- **LLM emotion extraction**: The example uses a placeholder `simple_emotion_extractor`. Swap it with your preferred LLM SDK (OpenAI, Google, etc.).

- **Audio input**: LiveKit will provide audio frames — wire them into `save_audio_bytes_bytes` or stream-write them. Implementation specifics depend on LiveKit SDK event hooks.
- **Safety**: Keep mental health disclaimers. This assistant is *not* a substitute for therapy — add a help message and crisis resources.
- **Extensions**:
- Add a small web UI to play audio clips and display mood graphs.
- Add scheduled nightly prompts using an external scheduler.

License

MIT